TEC | Tecnológico de Costa Rica

# Tarea I
# Diseño de sistema usando HDL

---


| | |
|---|---|
| **Institución:** | Instituto Tecnológico de Costa Rica |
| **Escuela:** | Ingeniería en Computadores |
| **Curso:** | Fundamentos de Arquitectura de Computadores (CE-1107) |
| **Profesor:** | Dr. Luis Alberto Chavarría |
| **Estudiantes:** | Alexander Angulo Serrano<br>Jose Manuel Chaves Chacón<br>Kendall Fernández Obando<br>Guillermo Sánchez Cedeño |
| **Semestre:** | II Semestre 2026 |
| **Fecha:** | 13 de octubre de 2026 |

# Índice

**1. Teoría** **1**
1.1. ¿Qué es un HDL? . . . 1
1.2. ¿Por qué un HDL como System Verilog tiene funciones que no se pueden sintetizar como por ejemplo *wait*? . . . 1
1.3. ¿Por qué no es correcto usar la palabra compilar (como un compilador) para HDLs? . . . 2
1.4. ¿Cuáles son tres ejemplos de lenguajes HDL? ¿Cuáles industrias los usan más habitualmente? . . . 2
1.4.1. VHDL . . . 2
1.4.2. Verilog . . . 2
1.4.3. SystemVerilog . . . 2
1.4.4. HDLs en semiconductores . . . 3
1.4.5. HDLs en telecomunicaciones . . . 3
1.4.6. HDLs en la industria espacial y de defensa . . . 3
1.5. ¿Qué es un modelo estructural y un modelo comportamental? ¿Por qué se le dice caja transparente al diseño estructural? Muestre una diferencia. . . . 3
1.5.1. Modelo estructural . . . 3
1.5.2. Modelo comportamental . . . 4
1.5.3. Ejemplo de diferencia entre ambos modelos . . . 4

**2. Elementos de diseño** **5**
2.1. Arquitectura general del sistema . . . 5
2.2. Bloques funcionales principales . . . 5
2.3. Diseño temporal y de sincronización . . . 6
2.4. Diseño de la máquina de estados . . . 6
2.5. Implementación en Quartus Prime . . . 7

# 1. Teoría

## 1.1. ¿Qué es un HDL?

Un lenguaje de descripción de hardware, HDL por sus siglas en inglés, es un lenguaje especializado para describir hardware, cuyo comportamiento es concurrente y no una secuencia de instrucciones. Permite describir las compuertas lógicas, las máquinas secuenciales y los módulos funcionales, así como su interconexión y control dentro de un sistema digital [1], de modo que una herramienta de diseño asistido por computadora (CAD) pueda producir una *netlist* con las compuertas optimizadas y sus conexiones, en un proceso llamado síntesis [2].

Además de la síntesis, otro de los propósitos fundamentales de los HDL es la simulación de la lógica, la cual permite verificar que los *inputs* aplicados a un módulo resulten en los *outputs* correctos. La simulación es importante porque permite corregir y debuggear los sistemas de hardware antes de construirlos [2].

## 1.2. ¿Por qué un HDL como System Verilog tiene funciones que no se pueden sintetizar como por ejemplo *wait*?

Un lenguaje de descripción de hardware (HDL), como SystemVerilog, permite describir circuitos digitales y también desarrollar modelos para simulación y verificación. Por esta razón, incluye construcciones que son útiles para representar el comportamiento durante una simulación, pero que no necesariamente corresponden a una estructura de hardware que una herramienta pueda sintetizar [3]

Un ejemplo es la sentencia `wait`, que suspende la ejecución de un proceso hasta que se cumple una condición. Por ejemplo, la instrucción `wait (ready);` detiene temporalmente dicho proceso de simulación hasta que la señal `ready` sea verdadera. Sin embargo, esta sentencia no representa por sí misma un componente físico específico, como una compuerta lógica, un multiplexor o un registro.

En un circuito físico, la lógica continúa operando de acuerdo con sus conexiones y señales de control; no ejecuta instrucciones secuenciales del mismo modo que un programa de software. Por lo tanto, para implementar una funcionalidad equivalente, normalmente se requiere describir explícitamente la lógica combinacional o secuencial correspondiente. La documentación de síntesis de Vivado indica que los controles temporales procedurales no están soportados por su sintetizador [4]

Es importante distinguir entre que una construcción sea válida en SystemVerilog y que sea sintetizable. Una sentencia puede utilizarse correctamente en un banco de pruebas (*testbench*) para controlar la simulación, sin que pueda emplearse directamente en una descripción RTL destinada a implementarse en una FPGA.

## 1.3. ¿Por qué no es correcto usar la palabra compilar (como un compilador) para HDLs?

La palabra *compilar* no es incorrecta en todos los contextos relacionados con los HDLs. El estándar IEEE 1800-2023 define la compilación como el proceso de analizar el código fuente y comprobar aspectos sintácticos y semánticos. También define la elaboración, que permite establecer la jerarquía de módulos, resolver parámetros y determinar sus interconexiones [3]

No obstante, utilizar *compilar* como sinónimo de convertir directamente una descripción HDL en hardware puede resultar impreciso. En un lenguaje de programación convencional, el compilador normalmente transforma el código fuente en una representación ejecutable por un procesador. En cambio, en el diseño digital, la síntesis interpreta la descripción del circuito, infiere los elementos lógicos necesarios y genera una representación estructural, por ejemplo, una lista de conexiones o *netlist*, que posteriormente puede mapearse a los recursos de un dispositivo programable [5]

Por consiguiente, es válido hablar de compilación cuando se hace referencia a la etapa de análisis del código SystemVerilog. Sin embargo, cuando se describe específicamente la transformación de una descripción RTL en hardware, el término más preciso es *síntesis*. En el flujo de diseño, la síntesis se distingue de las etapas posteriores de implementación física, como la ubicación y el enrutamiento de los recursos de la FPGA.

## 1.4. ¿Cuáles son tres ejemplos de lenguajes HDL? ¿Cuáles industrias los usan más habitualmente?

### 1.4.1. VHDL

El VHDL fue desarrollado originalemnte en 1981 por el Departamento de Defensa de EE.UU. para describir la estructura y funcionamiento del hardware. Tiene raíces en el lenguaje de programación de Ada. La IEEE lo estandarizó en 1987 (IEEE STD 1076). El VHDL primero se previó para documentar, pero finalmente se adoptó para los procesos de simulación y síntesis [2].

### 1.4.2. Verilog

Verilog es un HDL desarrollado en 1984 como lenguaje propietario para simulación de lógica. En 1990 se convirtió en un estándar abierto y cinco años después, en estándar de IEEE (IEEE STD 1364) [2].

### 1.4.3. SystemVerilog

SystemVerilog es un HDL utilizado para modelar, simular y verificar sistemas electrónicos. Surgió de la combinación de Verilog con Superlog y OpenVera. Superlog aportó sintaxis para aumentar la productividad en el diseño, como *struct* y *enum*, mientras que OpenVera mejoró la capacidad de expresar aserciones, lo que facilita la verificación en simulación. Además, incorporó la programación orientada a objetos, que se usa común-

mente en verificación. Fue estandarizado por primera vez en 2005 (IEEE STD 1800-2005) y su revisión de 2009 integró Verilog (IEEE STD 1364) como un subconjunto del lenguaje [6].

### 1.4.4. HDLs en semiconductores

Los HDL son fundamentales en las industrias de semiconductores y microelectrónica, pues cumplen un rol primordial en el diseño de los circuitos integrados de aplicación específica (ASIC) y procesadores [7]. Los ingenieros utilizan lenguajes como Verilog como una herramienta para describir la lógica de los circuitos antes de implementarla en el hardware [8]. Esto además permite indentificar los problemas en simulación, lo que previene construir semiconductores con errores de diseño y le permite a las compañías ahorrar millones de dólares [9].

### 1.4.5. HDLs en telecomunicaciones

En la industria de las telecomunicaciones, los HDL se utilizan ampliamente [10]. La creciente necesidad de transmitir datos de forma segura ha impulsado la implementación de algoritmos de encriptación directamente en hardware [10], y los HDL permiten diseñar estas soluciones para aplicaciones que requieren procesamiento de alta velocidad, como el procesamiento de datos y la encriptación de las comunicaciones en redes 5G [11]. Las *field-programmable gate arrays* (FPGA) son ampliamente utilizadas en la construcción de estas soluciones [12] [11], y para su diseño, la utilización de los HDL resulta fundamental [7].

### 1.4.6. HDLs en la industria espacial y de defensa

En las industrias espacial y de defensa los lenguajes de descripción de hardware también tienen un papel relevante, pues se utilizan fundamentalmente en la fabricación de ASICs y FPGAs especialemnte diseñados para utilizarse en satélites [13]. Además, como se mencionó anteriormente, el mismo VHDL fue desarrollado por el Programa de Circuitos Integrados de Gran Velocidad del Departamento de Defensa de EE.UU. [2] que tuvo como objetivo crea circuitos integrados de silicio de ultra alto rendimiento y alta densidad de señal para aplicaciones militares [14], así como diseñar componentes capaces de tolerar radiación ionizante y efectos de pulso electromagnético (EMP) en el espacio o en entornos de guerra [15].

## 1.5. ¿Qué es un modelo estructural y un modelo comportamental? ¿Por qué se le dice caja transparente al diseño estructural? Muestre una diferencia.

### 1.5.1. Modelo estructural

Un modelo estructural describe un circuito mediante la instanciación de componentes y la especificación explícita de sus conexiones. Estos componentes pueden ser compuertas

lógicas, módulos previamente definidos u otros bloques de hardware. El diseñador indica cuáles son los elementos que forman el circuito y cómo se interconectan entre sí [5]

Se denomina *caja transparente* o *caja blanca* porque su descripción permite observar cómo está construido internamente el circuito: se identifican sus componentes, las conexiones entre ellos y, cuando se incluyen, las señales intermedias. Esto facilita examinar la estructura y seguir el recorrido de las señales.

### 1.5.2. Modelo comportamental

Un modelo comportamental describe qué debe hacer el circuito a partir de sus entradas y de las condiciones establecidas, sin especificar necesariamente cada compuerta ni todas las conexiones internas. En SystemVerilog, puede expresarse mediante asignaciones, estructuras condicionales y bloques procedurales como `always_comb`.

A partir de esta descripción, una herramienta de síntesis puede inferir la estructura lógica que implementa el comportamiento especificado, siempre que se utilicen construcciones compatibles con la síntesis y que el diseño describa una función de hardware realizable ([5]).

### 1.5.3. Ejemplo de diferencia entre ambos modelos

Se puede ilustrar la diferencia utilizando un sumador completo, cuyas entradas son los bits `a`, `b` y `cin`, y cuyas salidas son `suma` y `cout`.

**Modelo estructural.** En este ejemplo se instancian explícitamente compuertas XOR, AND y OR. Las señales intermedias `t1`, `t2` y `t3` muestran cómo se conectan los componentes para generar el acarreo.

```
module sumador_estructural (
    input  wire a, b, cin,
    output wire suma, cout
);
    wire t1, t2, t3;

    xor (suma, a, b, cin);
    and (t1, a, b);
    and (t2, a, cin);
    and (t3, b, cin);
    or  (cout, t1, t2, t3);
endmodule
```

**Modelo comportamental.** En este caso se especifican las expresiones lógicas que determinan las salidas. No se instancian explícitamente las compuertas ni se declaran señales intermedias para construir el circuito.

```
module sumador_comportamental (
```

```
    input  logic a, b, cin,
    output logic suma, cout
);
    always_comb begin
        suma = a ^ b ^ cin;
        cout = (a & b) | (a & cin) | (b & cin);
    end
endmodule
```

Ambos modelos describen la misma función lógica. La diferencia principal está en el nivel de abstracción: el modelo estructural especifica los componentes y sus interconexiones, mientras que el comportamental expresa las relaciones lógicas que deben cumplirse entre las entradas y las salidas.

# 2. Elementos de diseño

## 2.1. Arquitectura general del sistema

El proyecto implementa un sistema digital de control de acceso basado en una secuencia de entrada configurable por medio de interruptores y una señal de activación por botón. La arquitectura se organiza en bloques funcionales claramente separados: entrada, validación, almacenamiento de la combinación, control de estado y salida visual. En el módulo superior `top.sv` se integran estos elementos mediante conexiones de señales que permiten coordinar la lógica combinacional con la lógica secuencial.

El diseño se apoya en una estructura jerárquica: los módulos pequeños describen funciones específicas y el módulo principal realiza la interconexión global del sistema. Este enfoque facilita la verificación individual de cada bloque y simplifica la depuración del comportamiento completo del circuito.

## 2.2. Bloques funcionales principales

- **Entrada y acondicionamiento de señales:** Los interruptores `SW[0:3]` representan las entradas del código, mientras que `KEY[0]` y los sensores de apertura/cierre de puerta (`SW_OPEN`, `SW_CLOSED`) se utilizan para activar la validación y la secuencia de operación del sistema. La señal de reloj principal, `CLOCK_50`, se deriva hacia un divisor de frecuencia para reducir la tasa de operación y sincronizar eventos del sistema.

- **Codificador:** El módulo `encoder` transforma la información de entrada en una representación comprimida de tres bits (`B0`, `B1`, `B2`). Esta etapa permite reducir la cantidad de señales que luego serán serializadas y evaluadas por el bloque de comprobación.

- **Validador de combinación:** El módulo `verifier` evalúa si la configuración de entradas cumple con una condición lógica de consistencia antes de aceptar la secuencia. Su lógica combina operaciones NOT, OR y AND para detectar entradas inválidas y avisar al resto del sistema cuando la combinación no es correcta.

- **Serialización y comprobación:** El módulo `serializer` toma los bits generados por el codificador y los desplaza mediante flip-flops para comparar la secuencia con una combinación esperada. El bloque `psw_checker` recibe las señales ya ordenadas y genera la señal `CORRECT`, que indica si la secuencia es válida.
- **Controlador de estados:** El módulo `state_controller` es el núcleo de la máquina de estados del sistema. Gestiona la condición de apertura y cierre de la puerta, además de generar señales de control como `OPENING`, `CLOSING`, `OPENED_out`, `E0` y `E1`. La lógica se apoya en un latch SR y en contadores basados en flip-flops para detectar eventos de transición y mantener estados coherentes.
- **Señales de salida:** El decodificador de anuncio enlaza los estados y genera la codificación para el display de siete segmentos (`HEX0`). Además, se conectan LEDs para mostrar el estado del proceso: apertura, cierre y validación incorrecta.

## 2.3. Diseño temporal y de sincronización

El sistema utiliza un reloj principal de 50 MHz que se convierte en una señal más lenta mediante el divisor `clock_divider`. Esta técnica es común en diseño digital porque evita que señales de control cambien demasiado rápido para ser observadas o procesadas por la lógica secuencial. El reloj lento permite que la máquina de estados avance en pasos perceptibles y evita problemas de ruido o de sincronización de señales de entrada.

La estructura del divisor usa flip-flops en configuración tipo ripple, en la que cada etapa depende del estado anterior. Esto genera una frecuencia de salida aproximada de 1.5 Hz, adecuada para pulsos de activación y para controlar la dinámica de apertura y cierre del sistema.

## 2.4. Diseño de la máquina de estados

La lógica de control no es un simple circuito combinacional: incluye memorias de estado y transiciones que dependen de señales externas como el botón de acción, la validación y el estado físico de la puerta. El módulo de control produce una secuencia lógica que representa los estados de:

- espera de ingreso,
- validación de la contraseña,
- apertura,
- cierre,
- señal de error por combinación inválida.

La salida `OPENING` indica que el sistema está ejecutando la apertura, mientras que `CLOSING` representa la fase de cierre. El uso de señales de estado explícitas facilita la depuración y permite documentar claramente la intención funcional del circuito.

## 2.5. Implementación en Quartus Prime

El diseño se implementó en Intel Quartus Prime Lite Edition sobre la tarjeta DE1-SoC, cuyo dispositivo es el Cyclone V 5CSEMA5F31C6. Cada bloque funcional se describió en un archivo SystemVerilog independiente y se instanció desde el módulo superior `top`. Las siguientes capturas muestran el código de cada módulo dentro del editor de Quartus, la jerarquía del proyecto en el *Project Navigator* y el resultado del análisis y elaboración, que terminó con 0 errores.

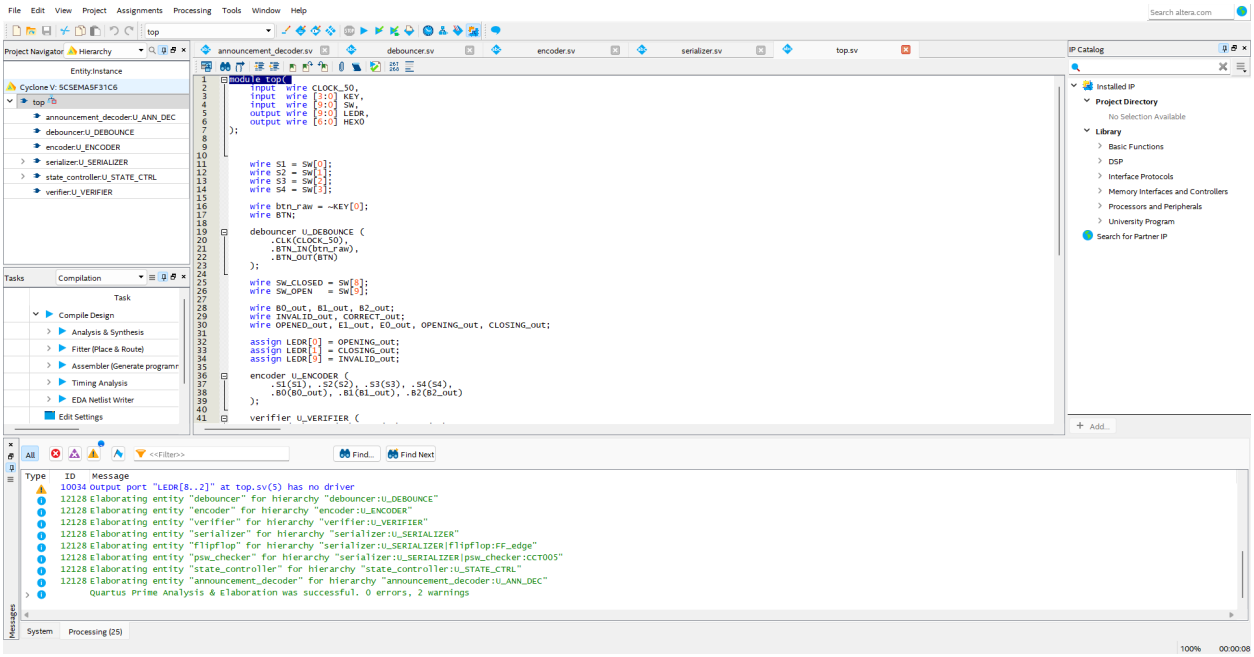



Figura 1: Módulo superior `top`: conexión de los interruptores, botones, LEDs y display de la DE1-SoC con los bloques del sistema.

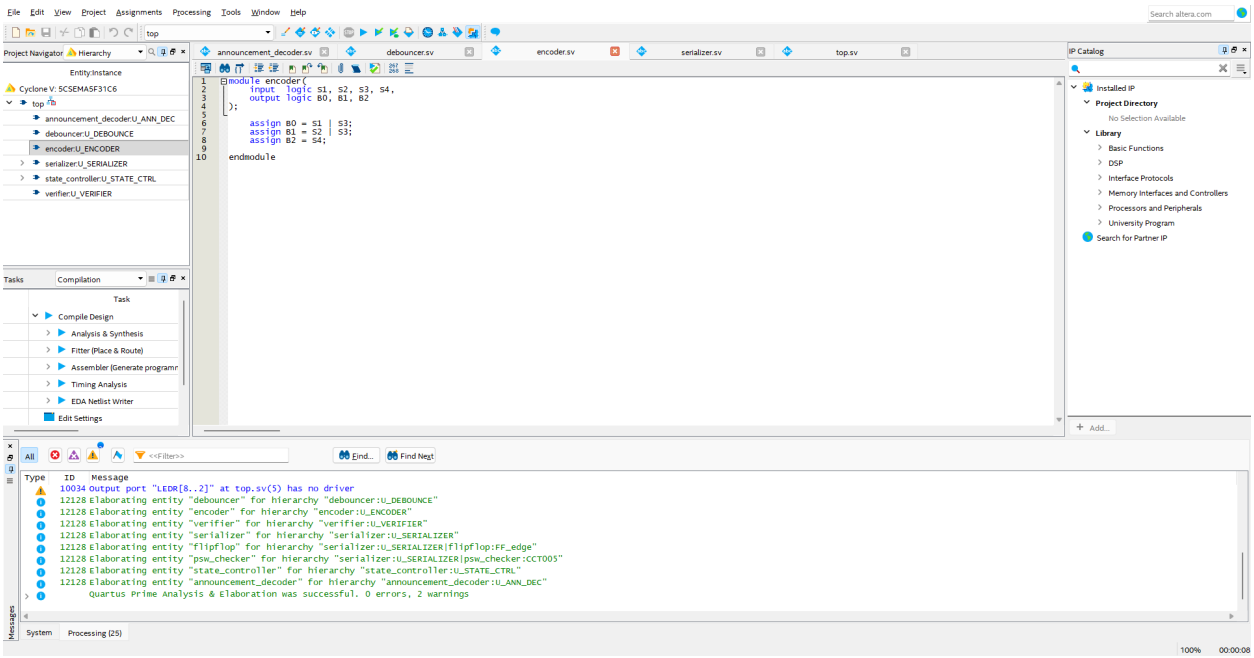



Figura 2: Módulo `encoder`: codificación de las señales de los sensores en los bits `B0`, `B1` y `B2`.

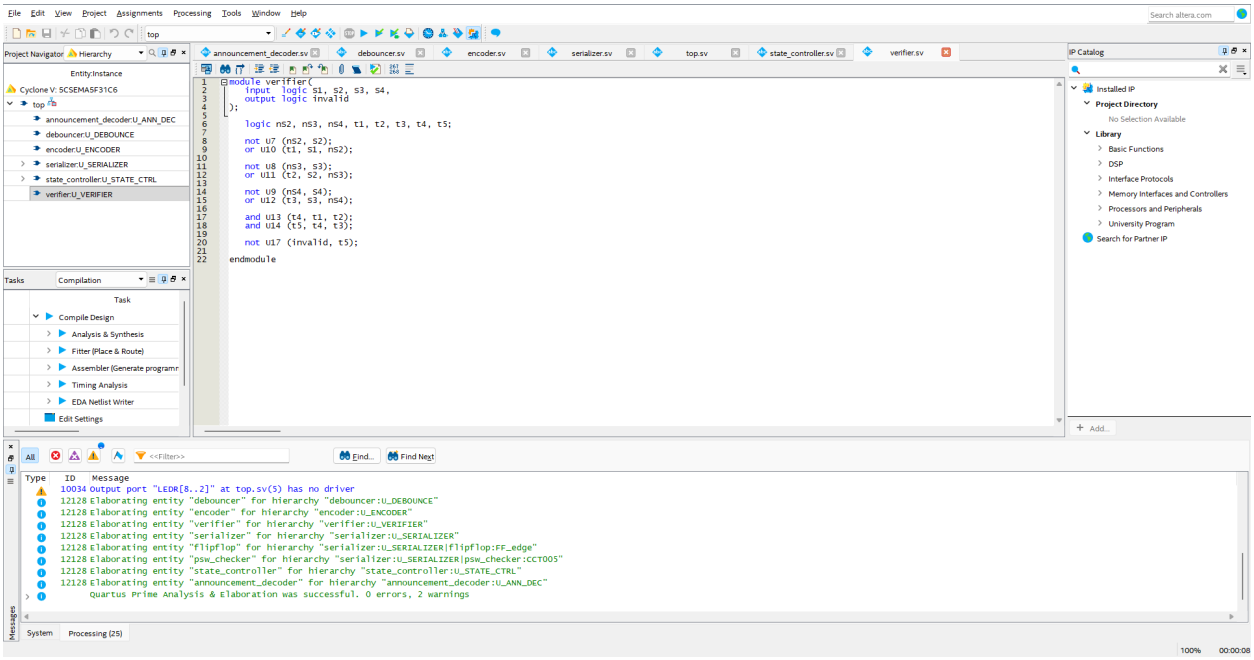



Figura 3: Módulo `verifier`: detección de combinaciones inválidas mediante compuertas NOT, OR y AND.

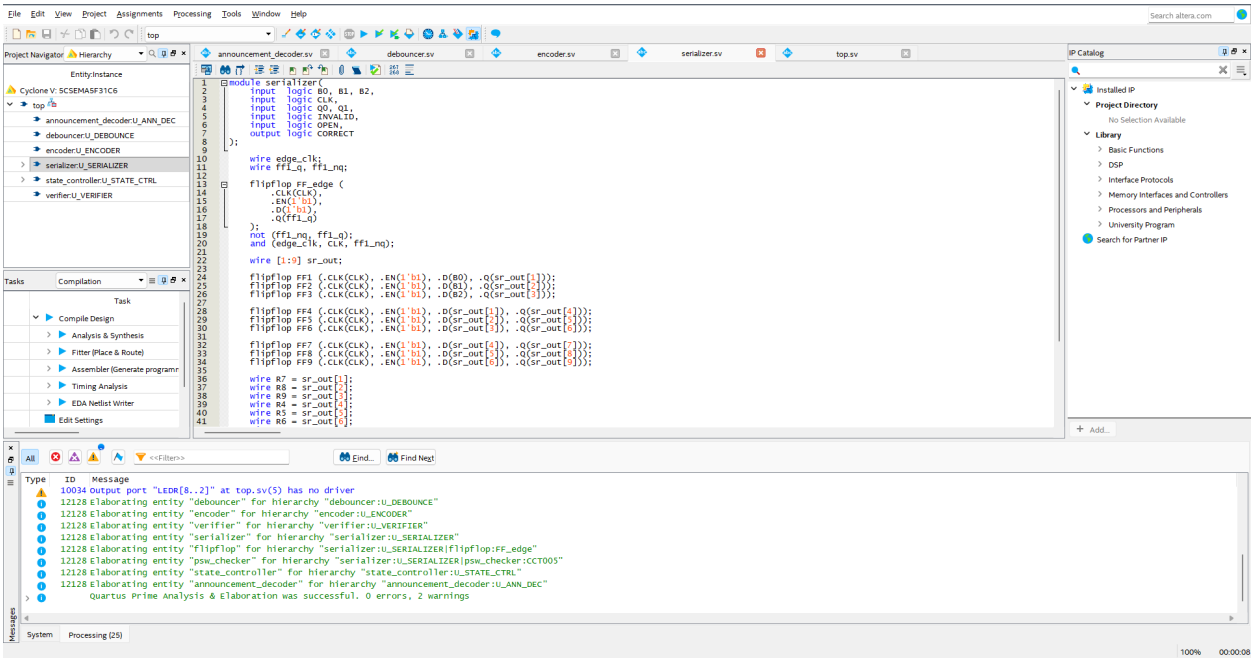



Figura 4: Módulo `serializer`: registro de desplazamiento de nueve etapas construido con instancias de `flipflop`.

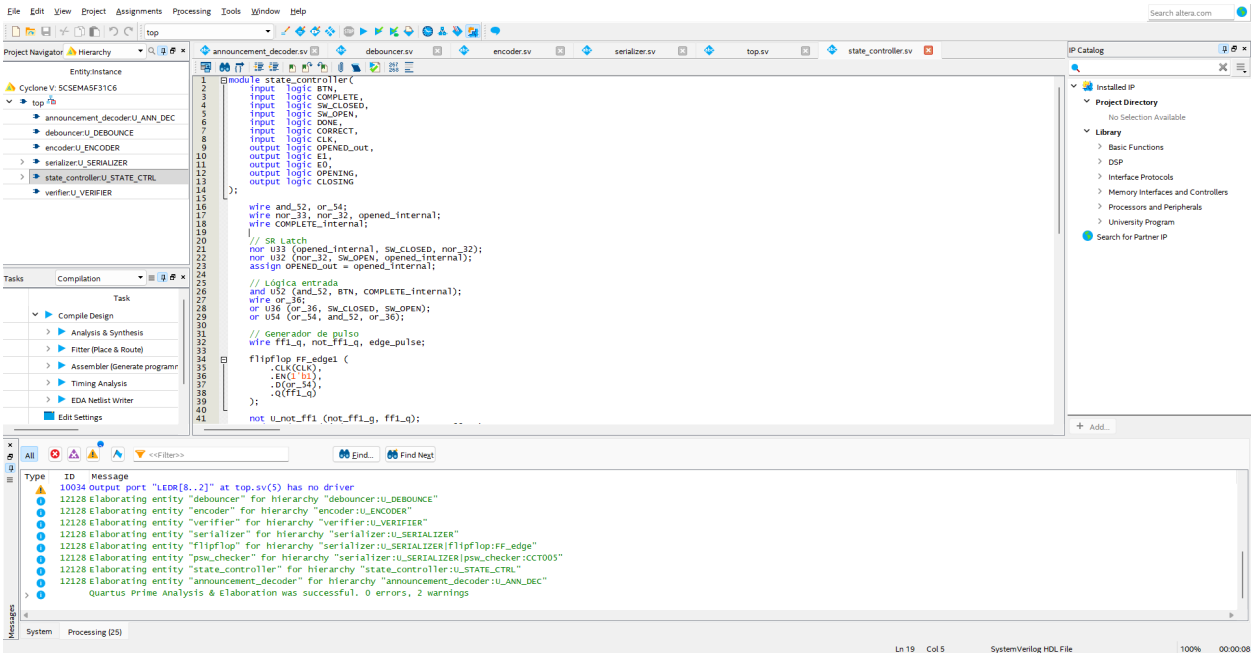



Figura 5: Módulo `state_controller`: latch SR con compuertas NOR, lógica de entrada y generador de pulso del controlador de estados.

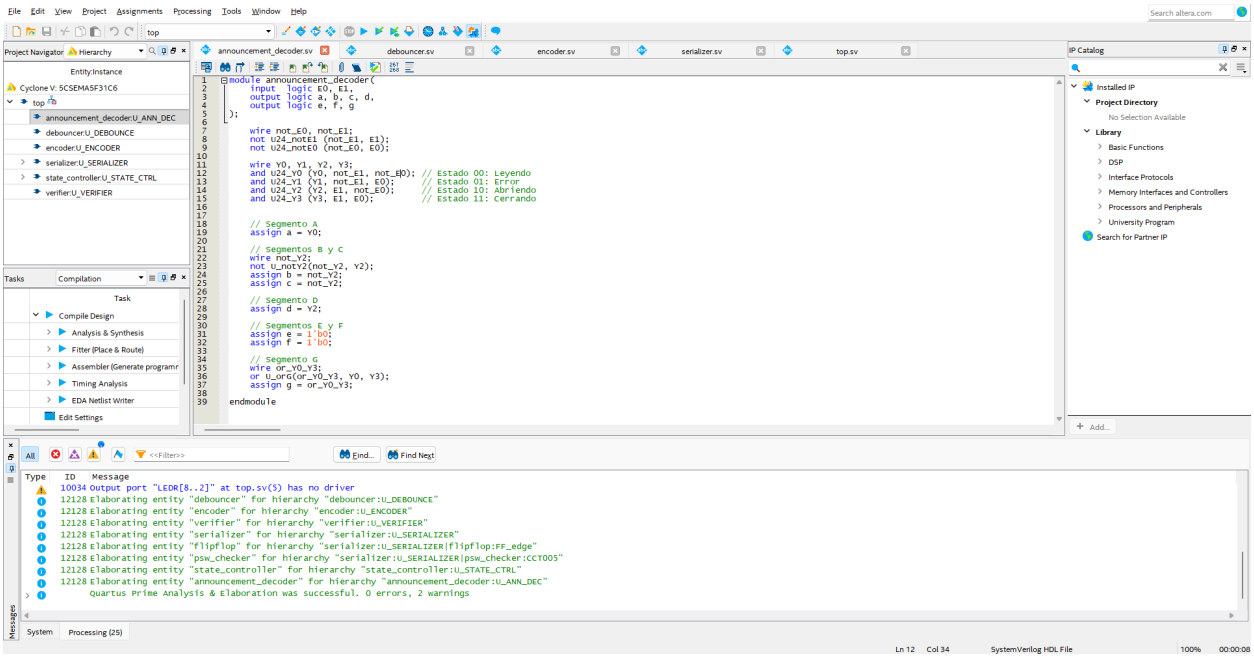



Figura 6: Módulo `announcement_decoder`: decodificación de los estados `E1 E0` a los segmentos del display `HEX0`.

# Referencias


[1] G. J. Lipovski, "Hardware description languages: voices from the tower of babel," *Computer*, vol. 10, no. 6, pp. 14–17, 1977.

[2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*. San Francisco, CA: Morgan Kaufmann, 2007.

[3] IEEE Computer Society, *IEEE standard for SystemVerilog—Unified hardware design, specification, and verification language*, IEEE.

[4] Advanced Micro Devices, Inc. Systemverilog constructs. [Online]. Available: https://docs.amd.com/r/en-US/ug901-vivado-synthesis/SystemVerilog-Constructs

[5] ——. Vivado design suite user guide: Synthesis (ug901). [Online]. Available: https://docs.amd.com/r/en-US/ug901-vivado-synthesis

[6] SystemVerilog, "SystemVerilog for RTL Modeling, Simulation, and Verification," https://systemverilog.dev/1.html, capítulo 1: Introduction. Consultado: 10-10-2026.

[7] D. J. Smith, *HDL Chip Design: A practical guide for designing, synthesizing and simulating ASICs and FPGAs using VHDL or Verilog*. Doone publications, 1998.

[8] S. A. Ansari A, A. Jones, M. Kumar S, M. Siva M, and A. F. I, "Ai-powered automated verilog generation and simulation for semiconductor design," in *2026 International Conference on Data Science and Business Systems (ICDSBS)*, vol. 3, 2026, pp. 1–5.

[9] S. L. Harris and D. Harris, "Hardware description languages," *Digital Design and Computer Architecture*, pp. 170–235, 2022.

[10] D. Saranyanandhini, T. Nivethitha, and M. MohanKumar, "Hdl implementation of des algorithm for telecommunication applications," in *2025 Third International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*. IEEE, 2025, pp. 1059–1064.

[11] P. Visconti, R. Velázquez, C. D.-V. Soto, and R. De Fazio, "Fpga based technical solutions for high throughput data processing and encryption for 5g communication: A review," *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 19, no. 4, pp. 1291–1306, 2021.

[12] A. Kumar, P. Kuchhal, S. Singhal, and A. Kumar, "Network on chip for dtmf decoder and tdm switching in telecommunication network with hdl environment," in *2013 3rd IEEE International Advance Computing Conference (IACC)*. IEEE, 2013, pp. 1582–1588.

[13] B.-H. Shim, Y.-J. Yu, and K.-M. Song, "Survey on advances in broadband signal generation and processing of asic semiconductors for defense satellites," *Journal of Electromagnetic Engineering and Science*, vol. 24, no. 4, pp. 377–383, 2024.

[14] C. Thornton, "The very high speed integrated circuit program," in *EASCON 1982; Annual Electronics and Aerospace Systems Conference*, 1982, pp. 297–299.

[15] IEEE, “Very high speed integrated circuits,” https://technav.ieee.org/topic/very-high-speed-integrated-circuits/, 2026, consultado: 11-10-2026.